# SystemDesigner: Current Development Documentation

May 23, 2026

## 1 Purpose

SystemDesigner is an early-stage Python toolkit for constructing atomistic crystal objects, materializing local object bases, and preparing higher-level assembly workflows. The main design principle is to keep local object geometry, global object arrangement, and additional physical fields separate.

The current project state is best understood as a layered system:

| Layer | Responsibility |
|---|---|
| CrystalDesigner | Build and operate on local crystal structures. |
| AssemblyBaseDesigner | Sample template parameters and materialize local objects. |
| AssemblyBaseStructurizer | Planned: create global object arrangements. |
| AssemblyBaseMagnetizer | Planned/in progress: attach local magnetic vector fields. |
| AssemblyBaseNucleizer | Planned: attach local nuclear SLD fields. |

## 2 Repository Layout

```
SystemDesigner/
    __init__.py

    CrystalDesigner/
        CrystalBase.py
        CrystalRepeater.py
        CrystalOperator.py
        CrystalTemplates.py
        CrystalPlot.py
        Example1.py

    AssemblyBaseDesigner/
        AssemblyBase.py
        AssemblyBaseTemplates.py
        AssemblyBaseWriter.py
        AssemblyBaseAnalyzer.py
        AssemblyBasePlot.py
        Example1.py

    AssemblyBaseMagnetizer/
        MagnetizationBase.py
        MagnetizationBaseTemplates.py
        SphericalVectorFieldLib.py
        AssemblyBaseMagnetizer.py
        __init__.py
```

```
    AssemblyBaseStructurizer/
    AssemblyBaseNucleizer/
```

# 3 CrystalDesigner

## 3.1 Data Model

The current crystal representation is a lightweight dictionary containing coordinate arrays and atom labels:

```
crystal = {
    "x": np.ndarray,
    "y": np.ndarray,
    "z": np.ndarray,
    "atomtype": np.ndarray,
}
```

Additional metadata may be present depending on how the crystal was generated. The structure is intentionally simple while the API is still evolving.

## 3.2 CrystalBase.py

`crystal_base(x, y, z, atomtype)` creates a clean base dictionary. It enforces one-dimensional coordinate arrays of equal length and broadcasts a single atom type label to all positions.

```
from SystemDesigner.CrystalDesigner.CrystalBase import crystal_base

base = crystal_base(
    x=0.0,
    y=0.0,
    z=0.0,
    atomtype="Fe",
)
```

## 3.3 CrystalRepeater.py

`crystal_repeater(...)` expands a crystal base by lattice translations:

$$\Delta r = n_1 a_1 u_1 + n_2 a_2 u_2 + n_3 a_3 u_3.$$

The generated crystal keeps mapping arrays such as `base_atom_index`, `n1`, `n2`, and `n3`.

## 3.4 CrystalOperator.py

Implemented operators:

- `crystal_centering`: center coordinates around their mean.
- `rotation_matrix_zyz`: construct an active Z-Y-Z Euler rotation matrix.
- `crystal_rotator_zyz`: rotate coordinate arrays `x`, `y`, and `z`.
- `crystal_rotate_zyz`: rotate a full crystal dictionary.
- `implicit_crystal_operator`: keep atoms satisfying an implicit inequality of the form `operator <= 0`.

The Z-Y-Z convention is

$$R = R_z(\alpha) R_y(\beta) R_z(\gamma),$$

where the matrix acts actively on column vectors. This can be used later to rotate raw crystals before applying a cutting operator, for example to generate random crystal orientations relative to a fixed cut geometry.

```
x_rot, y_rot, z_rot = crystal_rotator_zyz(
    x,
    y,
    z,
    alpha,
    beta,
    gamma,
)
```

Example spherical cut:

```
crystal = implicit_crystal_operator(
    crystal,
    "x**2 + y**2 + z**2 - R**2",
    {"R": 10.0},
)
```

### 3.5 CrystalTemplates.py

The current template registry contains:

```
CRYSTAL_TEMPLATE_REGISTRY = {
    "sc_sphere_crystal": sc_sphere_crystal,
}
```

`sc_sphere_crystal(a, R, atomtype)` creates a simple-cubic spherical crystal object by building a one-atom base, repeating it, centering it, and applying a spherical cut.

## 4 AssemblyBaseDesigner

### 4.1 Concept

AssemblyBaseDesigner does not place objects in global space. Instead, it defines and materializes local objects. This is the object library from which a later structuring or organizing layer can build a global assembly.

### 4.2 AssemblyBase.py

Assembly bases describe which crystal templates are available and how their parameters are sampled.

Supported parameter distributions:

- `constant(value)`
- `uniform(low, high)`
- `normal(mean, sigma)`
- `lognormal(mean, sigma, mean_type="arithmetic")`

Example:

```
from SystemDesigner.AssemblyBaseDesigner.AssemblyBase import (
    assembly_base, constant, normal
)

base = assembly_base(
    ct_temps="sc_sphere_crystal",
```

```
    ct_temps_params=["a", "R", "atomtype"],
    param_dist_props={
        "a": constant(1.0),
        "R": normal(mean=10.0, sigma=3.0),
        "atomtype": constant("Fe"),
    },
)
```

### 4.3 AssemblyBaseTemplates.py

The current high-level template is:

```
write_gaussian_spherical_nanoparticle_base(
    R_mean=10.0,
    R_std=3.0,
    a=1.0,
    atomtype="Fe",
    n_objects=500,
    output_dir="AssemblyExample1",
    seed=123,
)
```

This hides the underlying crystal template name `sc_sphere_crystal` from examples and user scripts.

### 4.4 AssemblyBaseWriter.py

The writer materializes sampled local objects to disk. The current output layout is:

```
RealSpaceData/
    Local_Objects/
        Object_1/
            pos.csv
            meta.csv
            atomtype.csv
        Object_2/
            ...
```

`pos.csv` contains local coordinates:

```
x,y,z
-7.0,0.0,0.0
...
```

`atomtype.csv` contains per-atom labels:

```
atom_index,atomtype
0,Fe
...
```

`meta.csv` contains object metadata and distribution metadata:

```
key,value
object_id,1
template_name,sc_sphere_crystal
n_atoms,1419
template_param.R,7.0326
template_param_distribution.R.distribution,normal
template_param_distribution.R.mean,10.0
template_param_distribution.R.sigma,3.0
```

### 4.5 AssemblyBaseAnalyzer.py

The analyzer reads `Local_Objects/Object_i/meta.csv` files and extracts realized template parameters. It also supports the legacy `RealSpaceData/Objects` layout.

Main functions:

- `analyze_assembly_base_dataset(path)`
- `write_parameter_table(analysis, output_path)`

### 4.6 AssemblyBasePlot.py

The plotter visualizes realized parameter distributions. Numeric parameters are shown as density histograms. If distribution metadata is available, the expected density is overlaid automatically for:

- normal distributions
- lognormal distributions
- uniform distributions
- constant values

## 5 AssemblyBaseMagnetizer

### 5.1 Current State

The magnetizer branch is in preparation. Implemented components are:

```
SystemDesigner/AssemblyBaseMagnetizer/MagnetizationBase.py
SystemDesigner/AssemblyBaseMagnetizer/MagnetizationBaseTemplates.py
SystemDesigner/AssemblyBaseMagnetizer/SphericalVectorFieldLib.py
```

The magnetizer is intended to operate on local object coordinates only. It should not sample global translations or global object rotations. Those belong to the structuring or organizing layer.

### 5.2 MagnetizationBase.py

`MagnetizationBase.py` mirrors the parameter-distribution logic of `AssemblyBaseDesigner`, but its semantics are local vector-field templates instead of crystal templates.

Supported distribution helpers are reused:

- `constant(value)`
- `uniform(low, high)`
- `normal(mean, sigma)`
- `lognormal(mean, sigma, mean_type="arithmetic")`

The central functions are:

- `magnetization_base(...)`
- `sample_field_params(...)`

### 5.3 MagnetizationBaseTemplates.py

Current high-level templates include:

- `spherical_vortex_magnetization_base`
- `spherical_random_chirality_vortex_magnetization_base`

These templates describe local field parameters only. For example, a local vortex template may define `field_type`, `profile_type`, `xi_type`, `kappa`, `turns`, and `chirality`.

## 5.4 SphericalVectorFieldLib.py

This module is a local analytical vector-field library. It does not read `Local_Objects`, and it does not write `MagData`. Higher-level magnetizer code should apply these fields to local object coordinates and store the resulting magnetic data separately.

The module currently contains two public building blocks:

- `alpha_profile(xi, params)`: computes a radial tilt angle $\alpha(\xi)$ from a normalized radius $\xi$.
- `unit_field(x, y, z, D, params)`: evaluates a normalized analytical vector field on local Cartesian coordinates.

The main field generator has the form:

```
unit_field(x, y, z, D, params) -> mx, my, mz
```

Here `D` is interpreted as the local particle diameter. The function derives either a cylindrical normalized radius $\xi = \rho/(D/2)$ or a spherical normalized radius $\xi = r/(D/2)$, depending on `xi_type`. Values are clipped to the interval $[0, 1]$. This makes the current library especially natural for spherical or radially parametrized particles.

The internal coordinate definitions are:

$$\rho = \sqrt{x^2 + y^2}, \qquad \phi = \operatorname{atan2}(y, x),$$
$$r = \sqrt{x^2 + y^2 + z^2}, \qquad \theta = \operatorname{atan2}\left(\sqrt{x^2 + y^2}, z\right).$$

Supported normalized-radius definitions are:

- `cylindrical_xi`: $\xi = \operatorname{clip}(\rho/R, 0, 1)$.
- `spherical_xi`: $\xi = \operatorname{clip}(r/R, 0, 1)$.

with $R = D/2$.

**Radial tilt profiles.** `alpha_profile` maps $\xi$ to a tilt angle $\alpha$. The current profile types are:

$$\begin{aligned}
\alpha_{\text{linear}}(\xi) &= \arccos(1 - \kappa\xi), \\
\alpha_{\text{parabolic}}(\xi) &= \arccos(1 - \kappa\xi^2), \\
\alpha_{\text{trigonometric}}(\xi) &= \frac{\pi}{2}\kappa\xi, \\
\alpha_{\text{hyperbolic}}(\xi) &= \arccos\left(\sqrt{1 - \tanh^2(\kappa\xi)}\right), \\
\alpha_{\text{argument_hyperbolic}}(\xi) &= \frac{\pi}{2}\tanh(\kappa\xi), \\
\alpha_{\text{lorentzian}}(\xi) &= \arccos\left((1 + \kappa\xi^2)^{-n}\right), \\
\alpha_{\text{gaussian}}(\xi) &= \arccos\left(\exp\left[-\xi^2/\kappa^2\right]\right), \\
\alpha_{\text{arctan}}(\xi) &= 2\arctan(\kappa\xi).
\end{aligned}$$

Here $\kappa$ controls the radial transition strength and $n$ is the `exponent` parameter used by the Lorentzian profile. In `unit_field`, most field types then use $\tilde{\alpha} = \texttt{turns} \cdot \alpha(\xi)$.

**Analytical field types.** `unit_field` currently supports the following local analytical field families:

**`linearized_vortex`.**

$$m_x = -m_1 y, \qquad m_y = m_1 x, \qquad m_z = m_0.$$

**vortex.** With $m_\phi = \sin(\tilde{\alpha})$ and $m_z^{(0)} = \cos(\tilde{\alpha})$:

$$m_x = -m_\phi \sin(\phi), \qquad m_y = m_\phi \cos(\phi), \qquad m_z = m_z^{(0)}.$$

**hedgehog.** With $m_r = \sin(\tilde{\alpha})\cos(\theta)$ and $m_z^{(0)} = \cos(\tilde{\alpha})$:

$$\begin{aligned} m_x &= m_r \sin(\theta)\cos(\phi), \\ m_y &= m_r \sin(\theta)\sin(\phi), \\ m_z &= m_r \cos(\theta) + m_z^{(0)}. \end{aligned}$$

**artichoke.** With $m_\theta = -\sin(\tilde{\alpha})$ and $m_z^{(0)} = \cos(\tilde{\alpha})$:

$$\begin{aligned} m_x &= m_\theta \cos(\phi)\cos(\theta), \\ m_y &= m_\theta \sin(\phi)\cos(\theta), \\ m_z &= -m_\theta \sin(\theta) + m_z^{(0)}. \end{aligned}$$

**poloidal_vortex.** With $m_\theta = \sin(\tilde{\alpha})$ and $m_z^{(0)} = \cos(\tilde{\alpha})$:

$$\begin{aligned} m_x &= m_\theta \cos(\phi)\cos(\theta), \\ m_y &= m_\theta \sin(\phi)\cos(\theta), \\ m_z &= m_\theta \sin(\theta) + m_z^{(0)}. \end{aligned}$$

**skyrmion.**

$$\begin{aligned} m_x &= \sin(\pi N \xi)\cos(m\phi + \gamma), \\ m_y &= \sin(\pi N \xi)\sin(m\phi + \gamma), \\ m_z &= \cos(\pi N \xi). \end{aligned}$$

Most field types use `profile_type`, `kappa`, `xi_type`, and `turns`. The skyrmion field instead uses the phase parameters `N`, `m`, and `gamma`. By default the resulting vectors are normalized before they are returned:

$$\mathbf{m} \leftarrow \frac{\mathbf{m}}{\sqrt{m_x^2 + m_y^2 + m_z^2} + 10^{-12}}.$$

The library should be treated as a field library, not as a magnetizer workflow. It contains analytical functions only. A later materialization layer can choose one of these field definitions, evaluate it on each object's local `pos.csv`, and write the resulting `x,y,z,mx,my,mz` data into `MagData`.

# 6 Planned Orthogonal Branches

## 6.1 AssemblyBaseStructurizer

The structuring layer should operate on already materialized `Local_Objects`. Its role is to generate global arrangement data, for example:

```
RealSpaceData/
    StructData.csv
```

This file may later contain global object centers, rotations, and object indices. It should not modify the local object data.

### 6.2 AssemblyBaseMagnetizer

The magnetizer should generate local magnetic data:

```
MagData/
    Object_1/
        m_1.csv
    Object_2/
        m_1.csv
```

with rows of the form:

```
x,y,z,mx,my,mz
```

### 6.3 AssemblyBaseNucleizer

The nucleizer should generate local nuclear SLD data without modifying the local object geometry.

## 7 Examples

### 7.1 AssemblyBaseDesigner Example

```
python SystemDesigner/AssemblyBaseDesigner/Example1.py
```

This generates:

```
AssemblyExample1/
    RealSpaceData/
        Local_Objects/
        parameter_table.csv
        parameter_distributions.png
```

### 7.2 CrystalDesigner Example

```
python SystemDesigner/CrystalDesigner/Example1.py
```

This builds and plots a simple-cubic Fe sphere. It opens a Matplotlib window.

## 8 Development Notes

- The project is still experimental and APIs may change.
- The current code uses structured dictionaries rather than dedicated Python classes.
- `gen_structure.py` is retained as a reference script, not as part of the cleaned SystemDesigner API.
- Runtime tests were performed with an external Pixi Python environment containing NumPy and Matplotlib.